

Asignatura: Diseño Interactivo

Laboratorios 13 y 14 — WebXR, Three.js y ARCore

LABORATORIO 13: Realidad Aumentada Passthrough con Escala de Habitación

1. Objetivo de aprendizaje

Al finalizar este laboratorio el alumno será capaz de:

- Distinguir conceptualmente entre Realidad Virtual (VR) y Realidad Aumentada (AR), y comprender cómo ambas coexisten en una misma sesión WebXR.
 - Iniciar una sesión `immersive-ar` con `local-floor` y `bounded-floor` usando la API nativa de WebXR.
 - Renderizar manualmente sobre un `XRWebGLLayer` sin depender del módulo automático de Three.js (`renderer.xr.enabled`).
 - Comprender la diferencia entre una matriz de transformación del viewer (`viewer-to-world`) y una matriz de vista (`view matrix`).
 - Anclar contenido 3D al piso físico de una habitación real usando ARCore.
 - Entender el papel del giroscopio, acelerómetro y magnetómetro del teléfono en el seguimiento 6DoF.
 - Implementar una escena donde el usuario camina libremente por su espacio físico mientras observa objetos virtuales anclados a escala real.
-

2. Fundamento teórico: ¿Cómo se juntan VR y AR en este laboratorio?

En el lenguaje común solemos separar VR y AR como mundos opuestos. Sin embargo, en la práctica técnica de WebXR, ambos comparten la misma infraestructura de renderizado, sensores y matemáticas espaciales. Este laboratorio explota precisamente esa continuidad.

2.1 Realidad Virtual (VR) en este contexto

Aunque estamos en una sesión *immersive-ar*, el modelo 3D (ciudad, río, árboles) constituye un **mundo virtual completo**. El usuario no está viendo una interfaz plana; está inmerso en un espacio tridimensional con:

- Geometría propia (modelo GLTF).
- Iluminación virtual (HemisphereLight, DirectionalLight).
- Materiales y texturas.
- Profundidad, oclusión y perspectiva.

La única diferencia con el VR tradicional es que el fondo no es un skybox opaco, sino la **cámara real del teléfono**. El mundo virtual se "pega" sobre el mundo real.

2.2 Realidad Aumentada (AR) passthrough

En modo passthrough, ARCore (el backend de Google) recibe el feed de la cámara trasera del celular y lo coloca en el framebuffer de WebXR **antes** de que Three.js dibuje cualquier cosa. Nuestro trabajo como desarrolladores es:

1. No borrar el color del framebuffer (para no tapar la cámara real).
2. Limpiar únicamente el buffer de profundidad (DEPTH_BUFFER_BIT), para que nuestros objetos 3D respeten la profundidad y no se dibujen uno encima del otro de forma incorrecta.
3. Renderizar la escena de Three.js sobre ese fondo ya existente.

El resultado: el usuario ve su sala real (suelo, paredes, muebles) con el modelo 3D flotando o apoyado sobre ella.

2.3 ¿Por qué esto es importante para Diseño Interactivo?

Como diseñadores interactivos, debemos entender que el usuario no distingue entre "real" y "virtual" cuando ambos ocupan el mismo espacio perceptivo. La coherencia espacial (que el modelo esté exactamente en el piso, que no flote ni se hunda de forma incongruente) es más importante que la fidelidad gráfica. Un modelo low-poly bien anclado genera más presencia que un modelo hiperrealista mal posicionado.

-
3. El giroscopio y los 6 grados de libertad (6DoF)

3.1 ¿Qué es 6DoF?

6 Degrees of Freedom (6 grados de libertad) significa que el sistema puede rastrear simultáneamente:

Grado	Eje	Descripción
Traslación X	Izquierda / Derecha	Movimiento lateral del usuario.
Traslación Y	Arriba / Abajo	Salto, agacharse, cambio de altura.
Traslación Z	Adelante / Atrás	Caminar hacia o lejos de un objeto.
Rotación X (Pitch)	Cabeceo	Inclinar el celular hacia arriba o abajo.
Rotación Y (Yaw)	Guiñada	Girar el celular a izquierda o derecha.
Rotación Z (Roll)	Alabeo	Inclinar el celular como volante.

3.2 ¿Qué sensores intervienen?

En un teléfono Android con ARCore, el seguimiento 6DoF no depende de un solo sensor. Es una **fusión sensorial** (sensor fusion):

- **Giroscopio (Gyroscope):** Mide la velocidad angular en los tres ejes. Es rápido y preciso para rotaciones, pero acumula error (drift) con el tiempo.
- **Acelerómetro:** Mide la aceleración lineal. Permite detectar la dirección de la gravedad (para saber dónde está el piso) y movimientos bruscos.
- **Magnetómetro:** Mide el campo magnético terrestre. Actúa como brújula digital, corrigiendo el drift del giroscopio en el eje Y (yaw).
- **Cámara trasera:** ARCore analiza los fotogramas en tiempo real usando SLAM (Simultaneous Localization and Mapping). Detecta puntos de interés visuales en el entorno (esquinas, texturas, bordes) y usa esa información para corregir la posición absoluta del dispositivo en el espacio.

3.3 ¿Por qué es importante para este laboratorio?

Sin 6DoF, el teléfono solo sabría hacia dónde apunta (3DoF, como un visor de cartón básico). No sabría si el usuario caminó dos pasos hacia adelante. ARCore proporciona 6DoF, lo que permite que el modelo permanezca anclado al piso mientras el usuario **camina físicamente alrededor de él**, viéndolo desde diferentes ángulos y distancias.

Esto transforma la experiencia de "mirar una pantalla" a "estar presente en el espacio". Esa es la diferencia entre un diseño interactivo plano y uno espacial.

4. Explicación del código función por función

4.1 HTML y CSS — Interfaz de inicio

```
<div id="overlay-inicio">

  <h2>Escala de habitacion</h2>

  <p id="indicaciones">...</p>

  <button id="btn-entrar">INICIAR EXPERIENCIA AR</button>

</div>
```

● Guía completa de Lab13 y Lab14 en ASCII



Aquí tienes las guías completas en texto ASCII. Puedes copiarlas y pegarlas directamente en tu documento de curso.

Universidad [Nombre de tu Institución]

Asignatura: Diseño Interactivo

Laboratorios 13 y 14 — WebXR, Three.js y ARCore

LABORATORIO 13: Realidad Aumentada Passthrough con Escala de Habitación

1. Objetivo de aprendizaje

Al finalizar este laboratorio el alumno será capaz de:

- Distinguir conceptualmente entre Realidad Virtual (VR) y Realidad Aumentada (AR), y comprender cómo ambas coexisten en una misma sesión WebXR.
- Iniciar una sesión `immersive-ar` con `local-floor` y `bounded-floor` usando la API nativa de WebXR.
- Renderizar manualmente sobre un `XRWebGLLayer` sin depender del módulo automático de Three.js (`renderer.xr.enabled`).

- Comprender la diferencia entre una matriz de transformación del viewer (`viewer-to-world`) y una matriz de vista (`view matrix`).
 - Anclar contenido 3D al piso físico de una habitación real usando ARCore.
 - Entender el papel del giroscopio, acelerómetro y magnetómetro del teléfono en el seguimiento 6DoF.
 - Implementar una escena donde el usuario camina libremente por su espacio físico mientras observa objetos virtuales anclados a escala real.
-

2. Fundamento teórico: ¿Cómo se juntan VR y AR en este laboratorio?

En el lenguaje común solemos separar VR y AR como mundos opuestos. Sin embargo, en la práctica técnica de WebXR, ambos comparten la misma infraestructura de renderizado, sensores y matemáticas espaciales. Este laboratorio explota precisamente esa continuidad.

2.1 Realidad Virtual (VR) en este contexto

Aunque estamos en una sesión `immersive-ar`, el modelo 3D (ciudad, río, árboles) constituye un **mundo virtual completo**. El usuario no está viendo una interfaz plana; está inmerso en un espacio tridimensional con:

- Geometría propia (modelo GLTF).
- Iluminación virtual (`HemisphereLight`, `DirectionalLight`).
- Materiales y texturas.
- Profundidad, oclusión y perspectiva.

La única diferencia con el VR tradicional es que el fondo no es un skybox opaco, sino la **cámara real del teléfono**. El mundo virtual se "pega" sobre el mundo real.

2.2 Realidad Aumentada (AR) passthrough

En modo passthrough, ARCore (el backend de Google) recibe el feed de la cámara trasera del celular y lo coloca en el framebuffer de WebXR **antes** de que Three.js dibuje cualquier cosa. Nuestro trabajo como desarrolladores es:

1. No borrar el color del framebuffer (para no tapar la cámara real).
2. Limpiar únicamente el buffer de profundidad (`DEPTH_BUFFER_BIT`), para que nuestros objetos 3D respeten la profundidad y no se dibujen uno encima del otro de forma incorrecta.
3. Renderizar la escena de Three.js sobre ese fondo ya existente.

El resultado: el usuario ve su sala real (suelo, paredes, muebles) con el modelo 3D flotando o apoyado sobre ella.

2.3 ¿Por qué esto es importante para Diseño Interactivo?

Como diseñadores interactivos, debemos entender que el usuario no distingue entre "real" y "virtual" cuando ambos ocupan el mismo espacio perceptivo. La coherencia espacial (que el modelo esté exactamente en el piso, que no flote ni se hunda de forma incongruente) es más importante que la fidelidad gráfica. Un modelo low-poly bien anclado genera más presencia que un modelo hiperrealista mal posicionado.

3. El giroscopio y los 6 grados de libertad (6DoF)

3.1 ¿Qué es 6DoF?

6 Degrees of Freedom (6 grados de libertad) significa que el sistema puede rastrear simultáneamente:

Grado	Eje	Descripción
Traslación X	Izquierda / Derecha	Movimiento lateral del usuario.
Traslación Y	Arriba / Abajo	Salto, agacharse, cambio de altura.
Traslación Z	Adelante / Atrás	Caminar hacia o lejos de un objeto.
Rotación X (Pitch)	Cabeceo	Inclinar el celular hacia arriba o abajo.
Rotación Y (Yaw)	Guiñada	Girar el celular a izquierda o derecha.
Rotación Z (Roll)	Alabeo	Inclinar el celular como volante.

3.2 ¿Qué sensores intervienen?

En un teléfono Android con ARCore, el seguimiento 6DoF no depende de un solo sensor. Es una **fusión sensorial** (sensor fusion):

- **Giroscopio (Gyroscope):** Mide la velocidad angular en los tres ejes. Es rápido y preciso para rotaciones, pero acumula error (drift) con el tiempo.
- **Acelerómetro:** Mide la aceleración lineal. Permite detectar la dirección de la gravedad (para saber dónde está el piso) y movimientos bruscos.
- **Magnetómetro:** Mide el campo magnético terrestre. Actúa como brújula digital, corrigiendo el drift del giroscopio en el eje Y (yaw).
- **Cámara trasera:** ARCore analiza los fotogramas en tiempo real usando SLAM (Simultaneous Localization and Mapping). Detecta puntos de interés visuales en el entorno (esquinas, texturas, bordes) y usa esa información para corregir la posición absoluta del dispositivo en el espacio.

3.3 ¿Por qué es importante para este laboratorio?

Sin 6DoF, el teléfono solo sabría hacia dónde apunta (3DoF, como un visor de cartón básico). No sabría si el usuario caminó dos pasos hacia adelante. ARCore proporciona 6DoF, lo que permite que el

modelo permanezca anclado al piso mientras el usuario **camina físicamente alrededor de él**, viéndolo desde diferentes ángulos y distancias.

Esto transforma la experiencia de "mirar una pantalla" a "estar presente en el espacio". Esa es la diferencia entre un diseño interactivo plano y uno espacial.

4. Explicación del código función por función

4.1 HTML y CSS — Interfaz de inicio

HTML

```
<div id="overlay-inicio">
  <h2>Escala de habitacion</h2>
  <p id="indicaciones">...</p>
  <button id="btn-entrar">INICIAR EXPERIENCIA AR</button>
</div>
```

¿Por qué existe el overlay? WebXR requiere un **gesto del usuario** (click, tap) para iniciar la sesión. No puedes llamar `navigator.xr.requestSession()` automáticamente al cargar la página. El overlay cumple esa función de seguridad del navegador y además da contexto al usuario. El CSS usa `position: absolute` con `z-index` para superponer la UI sobre el canvas. `touch-action: none` evita que el navegador intercepte gestos de scroll o zoom.

4.2 Imports y variables globales

```
import * as THREE from 'three';
```

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
```

Usamos **módulos ES6** con `importmap`. Esto evita bundlers y permite que el navegador resuelva las rutas directamente desde la carpeta `build/` y `jsm/`.

Variables clave:

Variable	Propósito
<code>camera</code> , <code>scene</code> , <code>renderer</code>	Núcleo de Three.js
<code>gl</code>	Contexto WebGL raw (necesario para <code>XRWebGLLayer</code>)
<code>xrSession</code>	Sesión WebXR activa
<code>xrRefSpace</code>	Espacio de referencia (<code>local-floor</code> o <code>bounded-floor</code>)
<code>xrGLLayer</code>	Capa de renderizado que conecta WebGL con WebXR
<code>modeloEscena</code>	El objeto 3D cargado (ciudad o fallback)
<code>boundsLine</code>	Líneas verdes que delimitan la habitación

4.3 `initThree()` — Construcción del mundo virtual

4.3.1 Cámara

```
camera = new THREE.PerspectiveCamera(60, window.innerWidth / window.innerHeight, 0.1, 1000);  
camera.matrixAutoUpdate = false;
```

Concepto crítico: `matrixAutoUpdate = false` le dice a Three.js: *"No recalculas la matriz de la cámara a partir de `position`, `rotation` y `scale`. Yo la controlaré manualmente frame a frame."*

Esto es indispensable porque **vamos a inyectar la matriz de vista de WebXR directamente**. Si dejamos `matrixAutoUpdate = true` (valor por defecto), Three.js sobrescribiría nuestro trabajo en el momento del render.

4.3.2 Contexto WebGL y XRWebGLLayer

```
gl = canvas.getContext('webgl2', { antialias: true })
```

```
|| canvas.getContext('webgl', { antialias: true });
```

```
await gl.makeXRCompatible();
```

`makeXRCompatible()` es una **promesa obligatoria**. Le dice al navegador: *"prepárate para usar este contexto WebGL con una sesión XR inmersiva"*. Sin esto, la construcción de `XRWebGLLayer` falla con el error:

"Failed to construct XRWebGLLayer: WebGL context must be marked as XR compatible"

4.3.3 Renderer

```
renderer = new THREE.WebGLRenderer({
```

```
  canvas: canvas,
```

```
  context: gl,
```

```
  antialias: true,
```

```
  alpha: true
```

```
});
```

```
renderer.autoClear = false;
```

```
scene.background = null;
```

Notas clave:

- **alpha: true** permite que el canvas sea transparente. ARCore dibuja la cámara real detrás; nosotros solo pintamos los objetos 3D encima.
- **autoClear = false** evita que Three.js borre el framebuffer completo antes de renderizar. Si lo borrara, taparíamos la imagen de la cámara real.
- **scene.background = null** elimina cualquier fondo de color o textura que Three.js pudiera pintar.

4.3.4 Grupo contenedor del mundo virtual

```
const worldGroup = new THREE.Group();  
worldGroup.position.set(0, 0, 0);  
scene.add(worldGroup);  
scene.userData.worldGroup = worldGroup;
```

`worldGroup` es un `THREE.Group` que contiene todo el contenido virtual. Mover este grupo es visualmente equivalente a mover todo el mundo 3D respecto al usuario. En este laboratorio lo mantenemos en $(0, 0, 0)$ para que el modelo quede exactamente sobre el piso real.

4.3.5 Iluminación

```
const hemiLight = new THREE.HemisphereLight(0xffffff, 0x444444, 2.5);  
const dirLight = new THREE.DirectionalLight(0xffffff, 2.0);
```

La `HemisphereLight` simula luz ambiental (cielo/suelo). La `DirectionalLight` proyecta sombras direccionales. Ambas son **baratas computacionalmente** y adecuadas para móvil.

4.3.6 Carga del modelo GLTF

```
const loader = new GLTFLoader();  
loader.load('models/gltf/camp/ciudad.gltf', (gltf) => {  
  modeloEscena = gltf.scene;  
  modeloEscena.scale.set(0.07, 0.07, 0.07); // Escala de maqueta  
  modeloEscena.traverse((node) => { ... });  
  worldGroup.add(modeloEscena);  
});
```

El loader es asíncrono. Si el modelo no se encuentra, el callback de error crea una **geometría de respaldo** (edificios en cuadrícula) para que la aplicación no falle silenciosamente.

La escala 0.07 (7%) proviene del pipeline de exportación desde Unity. A ese tamaño, la ciudad se ve como una **maqueta a escala de mesa** sobre el piso real de la sala.

4.4 `iniciarExperiencia()` — Negociación con WebXR

4.4.1 Detección de soporte

```
const supported = await navigator.xr.isSessionSupported('immersive-ar');
```

Verificamos que el navegador pueda crear una sesión AR. Si no es así, informamos al usuario en lugar de dejarlo con una pantalla en blanco.

Verificamos que el navegador pueda crear una sesión AR. Si no es así, informamos al usuario en lugar de dejarlo con una pantalla en blanco.

4.4.2 Sesión y espacio de referencia

```
xrSession = await navigator.xr.requestSession('immersive-ar', {  
  requiredFeatures: ['local-floor'],  
  optionalFeatures: ['bounded-floor']  
});
```

local-floor (requerido): El origen del sistema de coordenadas se coloca en el piso físico detectado por ARCore, aproximadamente debajo del usuario. El eje Y apunta hacia arriba.

● **bounded-floor** (opcional): Si el dispositivo lo soporta, entrega los límites físicos de la habitación como un polígono.

Fallback:

```
try {  
  const boundedRefSpace = await xrSession.requestReferenceSpace('bounded-floor');  
  // Dibujar límites verdes  
} catch (e) {  
  // Fallback a local-floor sin límites dibujados  
}
```

Esto replica exactamente la lógica del ejemplo WebXR original.

4.4.3 XRWebGLLayer

```
xrGILayer = new XRWebGLLayer(xrSession, gl);  
xrSession.updateRenderState({ baseLayer: xrGILayer });
```

XRWebGLLayer es el puente entre WebXR y WebGL. WebXR toma el control del framebuffer y decide dónde y cómo renderizar. Nosotros solo leemos el viewport que nos asigna.

4.5 onXRFrame () — El corazón del sistema

4.5.1 Obtener la pose del viewer

```
const pose = frame.getViewerPose(xrRefSpace);
```

```
const view = pose.views[0];
```

```
const viewport = xrGLLayer.getViewport(view);
```

`getViewerPose ( )` devuelve la posición y orientación del dispositivo en el espacio de referencia que elegimos (`local-floor`). En un teléfono móvil hay exactamente **una vista** (`views[0]`), a diferencia del VR estéreo que tiene dos (ojo izquierdo y derecho).

4.5.2 Enlazar el framebuffer

```
gl.bindFramebuffer(gl.FRAMEBUFFER, xrGLLayer.framebuffer);
```

```
gl.viewport(viewport.x, viewport.y, viewport.width, viewport.height);
```

WebXR maneja su propio framebuffer. Debemos enlazarlo explícitamente y respetar el viewport que nos asigna el compositor.

4.5.3 Aplicar la transformación del viewer

```
camera.matrixWorldInverse.fromArray(view.transform.inverse.matrix);
```

```
camera.matrixWorld.copy(camera.matrixWorldInverse).invert();
```

```
camera.matrix.copy(camera.matrixWorld);
```

Aquí ocurre la magia. `view.transform.inverse.matrix` es la **View Matrix** de WebXR: transforma coordenadas del mundo al espacio de la cámara. La copiamos directamente a `camera.matrixWorldInverse` de Three.js. Luego invertimos para obtener `matrixWorld` (la posición/orientación de la cámara en el mundo).

Esto significa que **la cámara de Three.js y el dispositivo físico están perfectamente sincronizados**. Si caminas, la cámara se traslada. Si giras, la cámara rota.

4.5.4 Limpieza y render

```
gl.clear(gl.DEPTH_BUFFER_BIT);
```

```
renderer.render(scene, camera);
```

`gl.clear(gl.DEPTH_BUFFER_BIT)`: Limpia solo el buffer de profundidad. No tocamos el color porque ARCore ya dibujó la cámara real.

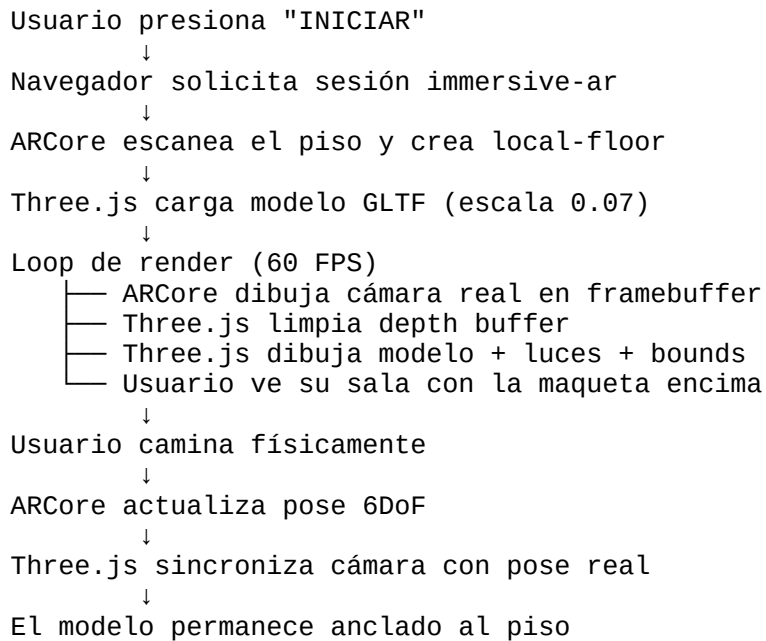
● `renderer.render(scene, camera)`: Three.js dibuja los objetos 3D sobre el fondo real.

4.6 dibujarLimites() — Visualización del espacio físico

```
function dibujarLimites(refSpace) {  
  const puntos = [];  
  const geom = refSpace.boundsGeometry;  
  for (let i = 0; i < geom.length; i++) {  
    const p1 = geom[i];  
    const p2 = geom[(i + 1) % geom.length];  
    puntos.push(new THREE.Vector3(p1.x, p1.y, p1.z));  
    puntos.push(new THREE.Vector3(p2.x, p2.y, p2.z));  
  }  
  const lineGeo = new THREE.BufferGeometry().setFromPoints(puntos);  
  const lineMat = new THREE.LineBasicMaterial({ color: 0x00ff00 });  
  boundsLine = new THREE.LineSegments(lineGeo, lineMat);  
  scene.userData.worldGroup.add(boundsLine);  
}
```

Cuando el dispositivo soporta `bounded-floor`, ARCore entrega los límites de la habitación como un polígono cerrado. Dibujamos ese polígono como líneas verdes (`0x00ff00`) para que el usuario vea visualmente hasta dónde puede caminar de forma segura.

5. Diagrama de flujo del sistema



Código QR

